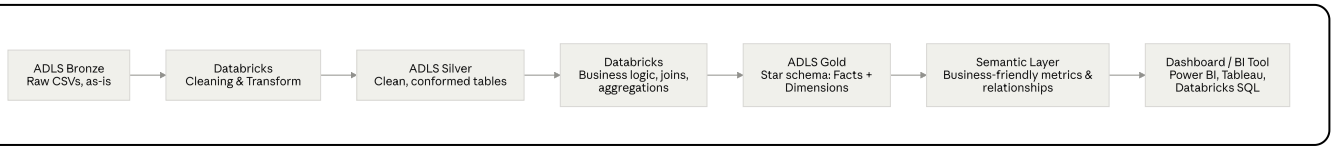
AdventureWorks on Databricks: Bronze → Silver → Gold → Semantic Layer → Dashboard

A step-by-step guide from the basics

---

0. The big picture first

You're building a **Medallion Architecture** — a standard pattern for organizing data as it moves through stages of increasing quality and business-readiness.



Layer	Purpose	Grain
Bronze	Raw, unaltered copy of source data (your 10 CSVs, as landed)	Same as source
Silver	Cleaned, deduplicated, correctly typed, conformed (one row = one real-world record)	Same as source, but trustworthy
Gold	Business-level, aggregated/modeled data — star schema, ready for reporting	Modeled for analysis (facts + dimensions)

You already have Bronze (raw CSVs landed in ADLS) and are doing "some transformation" into Silver. Let's make that precise.

---

1. Bronze Layer (what you already have)

Just the 10 files as-is, ingested into Delta tables with minimal/no changes — usually just:

- Add ingestion metadata: `_ingestion_date`, `_source_file`, `_batch_id`
- Store as **Delta format** (not CSV) even in Bronze, so you get ACID + schema tracking
- No business logic here — Bronze is your "raw, replayable" copy

```python
df = spark.read.option("header", True).csv("abfss://bronze@.../AdventureWorks_S
df = df.withColumn("_ingestion_date", current_timestamp()) \
       .withColumn("_source_file", input_file_name())
df.write.format("delta").mode("overwrite").save("abfss://bronze@.../sales_2017"
```

---

2. Silver Layer — transformations needed, table by table

This is the layer where you fix data quality issues and standardize types. Here's exactly what your files need, based on inspecting them:

2.1 Sales (2015, 2016, 2017 → one Silver table)

Issue found	Transformation
Split across 3 yearly files	`unionByName()` all three into one `silver_sales` table
`OrderDate`, `StockDate` are strings (`1/1/2017`)	Cast to `DateType` using `to_date(col, 'M/d/yyyy')`
No unique row identifier	Create surrogate key: `OrderNumber + OrderLineItem`
`OrderQuantity` is a string	Cast to `IntegerType`
No revenue/cost columns yet	These come later, in Gold, by joining to Products

python

```python
from pyspark.sql.functions import to_date, col, concat_ws

sales_15 = spark.read.format("delta").load(".../bronze/sales_2015")
sales_16 = spark.read.format("delta").load(".../bronze/sales_2016")
sales_17 = spark.read.format("delta").load(".../bronze/sales_2017")

sales = sales_15.unionByName(sales_16).unionByName(sales_17)

sales_silver = (sales
    .withColumn("OrderDate", to_date(col("OrderDate"), "M/d/yyyy"))
    .withColumn("StockDate", to_date(col("StockDate"), "M/d/yyyy"))
    .withColumn("OrderQuantity", col("OrderQuantity").cast("int"))
    .withColumn("SalesOrderLineKey", concat_ws("-", col("OrderNumber"), col("Or
    .dropDuplicates(["SalesOrderLineKey"])
)
```

2.2 Returns

Issue found	Transformation
`ReturnDate` is a string	Cast to `DateType`
`ReturnQuantity` is a string	Cast to `IntegerType`
No returns line key	Create surrogate key if you need row-level uniqueness

2.3 Customers (18,148 rows)

Issue found	Transformation
130 nulls in `Prefix`	Fill with `"N/A"` or leave null — low business impact
130 nulls in `Gender`	Decide policy: impute `"U"` (unknown) or leave null — **don't silently drop rows**
`BirthDate` is a string	Cast to `DateType`; optionally derive `Age`
`AnnualIncome` is text like `"$90,000"`	Strip `$`, `,`, trim, cast to `DecimalType`
`FirstName`/`LastName` inconsistent case	Standardize with `initcap()`
Free-text `Occupation`, `EducationLevel`	Trim whitespace, standardize casing

python

```
from pyspark.sql.functions import regexp_replace, trim, initcap

customers_silver = (customers_bronze
    .withColumn("BirthDate", to_date(col("BirthDate"), "M/d/yyyy"))
    .withColumn("AnnualIncome", regexp_replace(col("AnnualIncome"), "[$,]", "")
    .withColumn("FirstName", initcap(trim(col("FirstName"))))
    .withColumn("LastName", initcap(trim(col("LastName"))))
    .withColumn("Gender", when(col("Gender").isNull(), "U").otherwise(col("Gend
    .dropDuplicates(["CustomerKey"])
)
```

2.4 Products

Issue found	Transformation
50 nulls in `ProductColor`	Fill with `"N/A"` (common AdventureWorks convention)
`ProductCost`, `ProductPrice` need validation	Cast to `Decimal`, check `Price >= Cost` (flag anomalies)
`ProductSize`/`ProductStyle` have placeholder `0` values	Normalize to `null` or `"N/A"` where `0` means "not applicable"

2.5 Product Subcategories / Categories

- Small reference tables — mainly just type casting and dedup on key columns. Low effort.

2.6 Territories

- Same — small reference/lookup table, validate `SalesTerritoryKey` is unique, trim text fields.

2.7 Calendar

Issue found	Transformation
Single `Date` column, string type	Cast to `DateType`
No derived attributes yet	Add `Year`, `Quarter`, `Month`, `MonthName`, `DayOfWeek`, `WeekOfYear`, `IsWeekend`, fiscal year if needed

python

```python
from pyspark.sql.functions import year, quarter, month, date_format, dayofweek

calendar_silver = (calendar_bronze
    .withColumn("Date", to_date(col("Date"), "M/d/yyyy"))
    .withColumn("Year", year("Date"))
    .withColumn("Quarter", quarter("Date"))
    .withColumn("Month", month("Date"))
    .withColumn("MonthName", date_format("Date", "MMMM"))
    .withColumn("DayOfWeek", date_format("Date", "EEEE"))
)
```

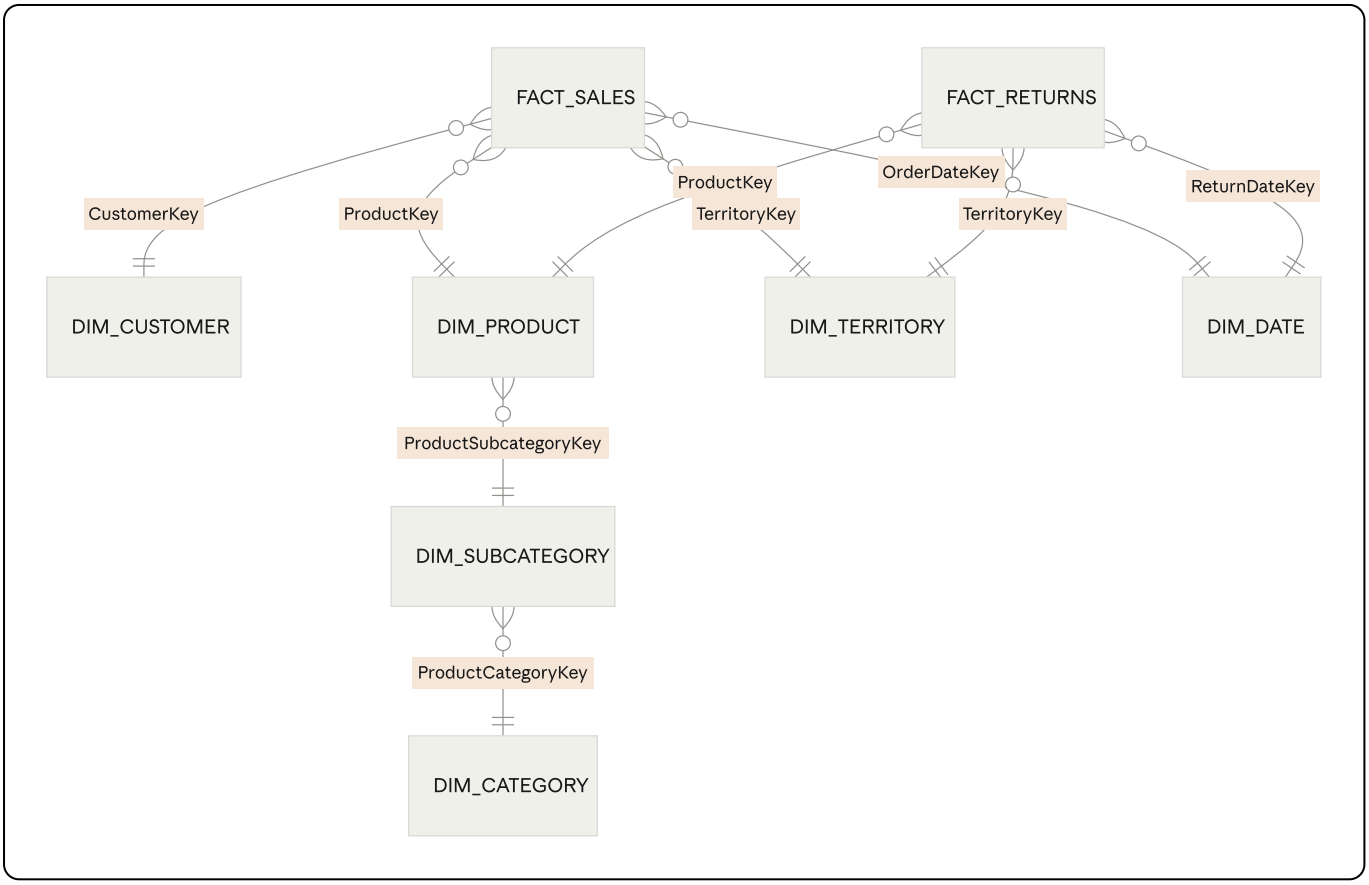
General Silver-layer rules of thumb

- **One notebook/job per table**, each idempotent (re-runnable without duplicating data) — use `MERGE INTO` for incremental loads.
- Enforce **schema** explicitly (don't rely on inferSchema in production).
- Add **data quality checks**: row counts, null-rate thresholds, key uniqueness (Databricks has built-in **Delta Live Tables (DLT) expectations** great for this).
- Write as **Delta tables**, partitioned where useful (e.g. Sales by `Year`).

---

3. Gold Layer — what it should contain

Gold is where you go from "clean tables" to a **star schema** designed for reporting. This is the biggest mindset shift: Silver = correctness, Gold = business usability.



Recommended Gold tables

`gold_fact_sales` (grain: one row per order line)

- Keys: `SalesOrderLineKey`, `OrderNumber`, `OrderLineItem`, `ProductKey`, `CustomerKey`, `TerritoryKey`, `OrderDateKey`, `StockDateKey`
- Measures: `OrderQuantity`, `UnitPrice` (from Products), `UnitCost`, `ExtendedRevenue = Qty × Price`, `ExtendedCost = Qty × Cost`, `GrossProfit = Revenue − Cost`

`gold_fact_returns` (grain: one row per return line)

- Keys: `ProductKey`, `TerritoryKey`, `ReturnDateKey`
- Measures: `ReturnQuantity`, and optionally `ReturnValue` (joined back to product price)

`gold_dim_customer` — cleaned customer attributes + derived `Age`, `AgeGroup`, `Full Name`

`gold_dim_product` — Products **joined** with Subcategory and Category, so it's a single denormalized dimension (`ProductName`, `SubcategoryName`, `CategoryName`, `Cost`, `Price`, `Margin%`)

`gold_dim_territory` — Region, Country, Continent

`gold_dim_date` — the enriched Calendar table

```python
gold_fact_sales = (sales_silver
    .join(products_silver.select("ProductKey", "ProductPrice", "ProductCost"),
    .withColumn("ExtendedRevenue", col("OrderQuantity") * col("ProductPrice"))
    .withColumn("ExtendedCost", col("OrderQuantity") * col("ProductCost"))
    .withColumn("GrossProfit", col("ExtendedRevenue") - col("ExtendedCost"))
)

gold_dim_product = (products_silver
    .join(subcategories_silver, "ProductSubcategoryKey", "left")
    .join(categories_silver, "ProductCategoryKey", "left")
    .withColumn("MarginPct", (col("ProductPrice") - col("ProductCost")) / col("
)
```

Why model it this way (star schema)?

- **Facts** = things that happened (sales, returns) — numeric, additive measures.
- **Dimensions** = the "who/what/where/when" context — text attributes you filter/group by.
- This shape is exactly what BI tools (Power BI, Tableau) and SQL analysts expect — fast joins, intuitive filtering, easy aggregation.

---

4. Semantic Layer — do you need one, and where?

A **semantic layer** sits on top of Gold and defines: business-friendly names, pre-built relationships, calculated measures (with correct aggregation logic), and access control — so anyone querying doesn't need to know the underlying SQL/joins.

You have three realistic options:

Option	What it is	When to use it
A. Power BI / Tableau Semantic Model	Import Gold tables into Power BI, define relationships + DAX measures in a dataset	Most common. Best if business users will self-serve in Power BI/Tableau.
B. Databricks Unity Catalog Metric Views	Define metrics/dimensions as governed objects directly in Unity Catalog, queryable from any tool via SQL	Best if you want **one governed definition of "Revenue"** usable across Databricks SQL, Power BI, and other tools without re-defining it each time.
C. dbt Semantic Layer (if you use dbt)	Metrics defined in dbt, served via dbt Semantic Layer API	Only relevant if your transformation pipeline is already dbt-based.

My recommendation for your case: since you're already on Databricks + ADLS, build your **Gold star schema in Delta tables**, then define a small set of **Unity Catalog Metric Views** (or a well-documented Gold schema) for your core metrics (Revenue, Gross Profit, Return Rate, Units Sold). This becomes your single source of truth regardless of which BI tool downstream users pick later.

---

5. Dashboard: Databricks SQL directly, or a BI tool?

This depends on your audience — here's how to decide:

Criteria	Databricks SQL Dashboard	Power BI / Tableau
Audience	Technical users, data teams, internal ops	Business users, execs, wide distribution
Setup effort	Low — query Gold tables directly with SQL, build dashboard in-platform	Medium — needs a semantic model, refresh schedule, licensing
Interactivity	Good (filters, parameters) but less polished	Rich (drill-through, bookmarks, mobile apps, natural-language Q&A)
Governance/sharing	Within Databricks workspace access	Broad org-wide sharing, row-level security, mobile
Cost	Included with Databricks SQL warehouse compute	Separate Power BI/Tableau licensing
Best for	Quick internal validation, ops monitoring, data-team dashboards	Company-wide reporting, exec dashboards, polished UX

Recommendation:

- Start with a **Databricks SQL Dashboard** directly on your Gold tables — it's fast to build, validates your model, and needs no extra tooling. Good for your first iteration and technical stakeholders.
- If this needs to go to business users / leadership, connect **Power BI (or Tableau)** to your Gold layer (via Databricks SQL Warehouse connector, which supports live/DirectQuery or Import mode) and build the polished semantic model + dashboard there.
- You don't have to choose only one — many teams do **both**: Databricks SQL for internal/ops monitoring, Power BI for exec/company-wide reporting — both pointing at the same Gold tables, so there's no logic duplicated.

---

6. Your step-by-step action plan

1. **Bronze**: Land all 10 CSVs as Delta tables in ADLS Bronze with ingestion metadata. *(Done)*
2. **Silver**: For each table, apply the type-casting, null-handling, and dedup transformations in Section 2. Write as Delta tables to ADLS Silver.
3. **Gold**: Union the 3 sales years, join Products↔Subcategory↔Category, compute Revenue/Cost/Profit measures, build the star schema (`gold_fact_sales`, `gold_fact_returns`, and the 5 dimension tables) in Section 3.
4. **Validate**: Run row-count and null-rate checks; confirm no orphaned foreign keys (e.g. every `ProductKey` in Sales exists in `dim_product`).
5. **Semantic layer**: Define your core business metrics once — either as Unity Catalog Metric Views or as a documented Power BI dataset — so "Revenue" means the same thing everywhere.
6. **Dashboard v1**: Build a Databricks SQL dashboard on Gold tables (Total Revenue, Revenue by Category/Territory/Time, Return Rate, Top Customers) to sanity-check the model.
7. **Dashboard v2 (if needed)**: Connect Power BI/Tableau to Gold for a polished, sharable version for business users.
8. **Automate**: Schedule the Bronze→Silver→Gold pipeline as a Databricks Job (or Delta Live Tables pipeline) so it refreshes on a cadence.